

EDA and Model Evaluation Metrics

KJR

- **1. Exploratory Data Analysis**
- **2. Model Evaluation**
 - Regularization in Machine Learning
 - Confusion Matrix
 - Precision, Recall and F1-Score
 - Bias and variance trade-off
 - **K-fold Cross-validation**
 - Hyperparameter Tuning & Fine Tuning

Exploratory Data Analysis

Exploratory Data Analysis (EDA) is an important step in data analysis where we explore, summarize, and visualize data to understand its structure, detect patterns, identify anomalies, test assumptions, and check relationships between variables before applying any machine learning or statistical models.

Exploratory Data Analysis (EDA)



EDA

Explore and summarize data before modeling



Distribution

Use histograms and box plots



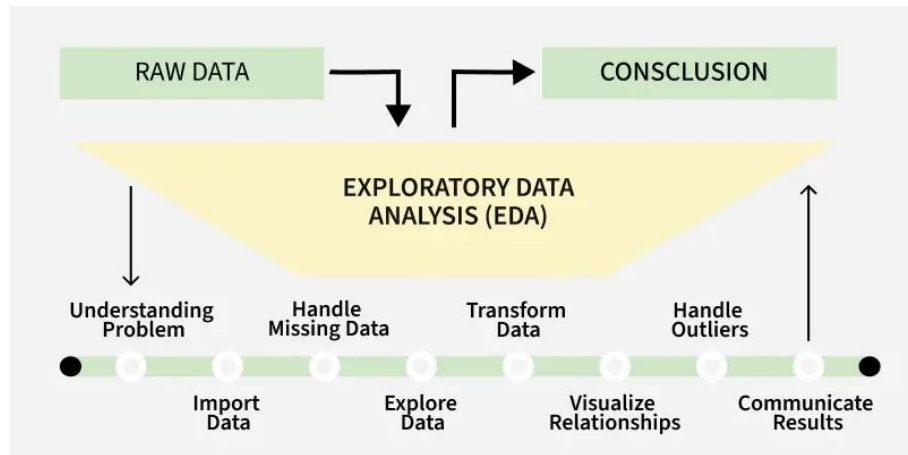
Relationships

Use scatter and pair plots



Spot Issues

Find outliers or missing data



Importance

- Provides a clear understanding of the dataset, including the number of features, data types and data distribution.
- Reveals patterns and relationships between different variables in the data.
- Identifies errors and outliers that may affect analysis.
- Highlights the most important features useful for building models.
- Supports selecting suitable modelling techniques for better results.

1. Univariate Analysis

Univariate analysis studies one variable at a time to understand its characteristics and distribution.

- **Histograms:** Show how data values are distributed.
- **Box plots:** Help detect outliers and show data spread.
- **Bar charts:** Used for categorical variables.

2. Bivariate Analysis

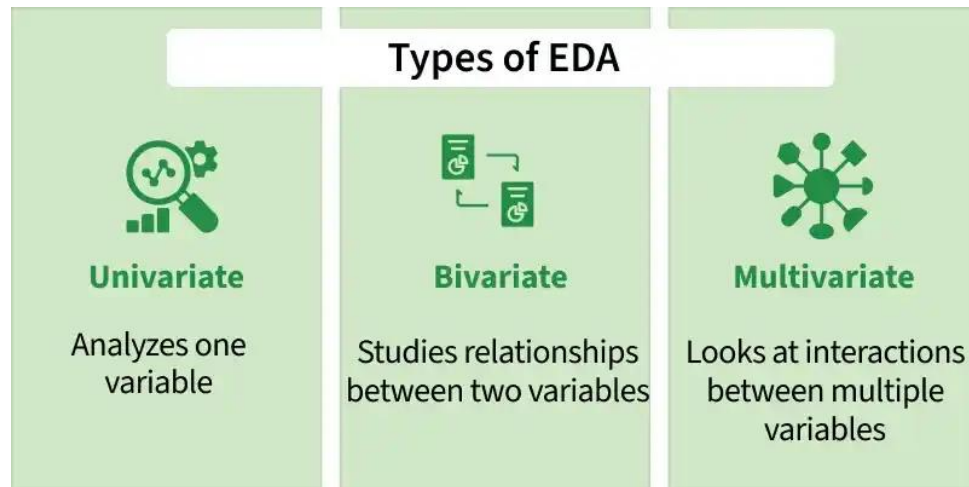
Bivariate analysis examines the relationship between two variables to understand how they interact or influence each other. Common techniques include:

- **Scatter plots:** Show the relationship between two numerical variables.
- **Correlation coefficient:** Measures the strength of the relationship between variables.
- **Cross-tabulation:** Displays the relationship between two categorical variables.
- **Line graphs:** Compare two variables over time to identify trends.
- **Covariance:** Shows how two variables change together.

3. Multivariate Analysis

Multivariate analysis studies three or more variables together to understand complex relationships within the dataset. Common techniques include:

- **Pair plots:** Show relationships between multiple variables at once.
- **Principal Component Analysis (PCA):** Reduces dimensionality while preserving important information.
- **Spatial analysis:** Analyzes geographical patterns using maps and location-based data.



Steps for Performing Exploratory Data Analysis

01

**Understanding
the Problem
and the Data**

02

**Importing and
Inspecting the
Data**

03

**Handling
Missing Data**

04

**Exploring Data
Characteristics**

05

**Performing Data
Transformation**

06

**Visualizing
Relationship of
Data**

07

**Handling
Outliers**

08

**Communicate
Findings and
Insights**

EDA - Exploratory Data Analysis in Python

EDA is an essential step in data analysis that focuses on understanding patterns, relationships and distributions within a dataset using statistical methods and visualizations. Python libraries such as pandas, NumPy, plotly, matplotlib and seaborn make this process efficient and insightful. Some common EDA techniques are:

- Data Inspection:** Check the size of the dataset, how it is organized, the types of data it contains and basic summary values.
- Handling Missing and Duplicate Data:** Find and fix empty values or repeated rows to keep the data clean.
- Univariate Analysis:** Study one variable at a time to understand its distribution, trend and outliers.
- Bivariate Analysis:** Compare two variables to see how they are related.
- Multivariate Analysis:** Analyze three or more variables together to understand deeper relationships.

Key Steps for Exploratory Data Analysis (EDA)

Step 1: Importing Required Libraries

We need to install [Pandas](#), [NumPy](#), [Matplotlib](#) and [Seaborn](#) libraries in python to proceed further.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings as wr
wr.filterwarnings('ignore')
```

Step 2: Reading Dataset

Let's read the dataset using pandas.

Download the dataset from this [link](#)

```
df = pd.read_csv("/content/WineQT.csv")
print(df.head())
```

Output:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	\
0	7.4	0.70	0.00	1.9	0.076	
1	7.8	0.88	0.00	2.6	0.098	
2	7.8	0.76	0.04	2.3	0.092	
3	11.2	0.28	0.56	1.9	0.075	
4	7.4	0.70	0.00	1.9	0.076	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
0	11.0	34.0	0.9978	3.51	0.56	
1	25.0	67.0	0.9968	3.20	0.68	
2	15.0	54.0	0.9970	3.26	0.65	
3	17.0	60.0	0.9980	3.16	0.58	
4	11.0	34.0	0.9978	3.51	0.56	

	alcohol	quality	Id
0	9.4	5	0
1	9.8	5	1
2	9.8	5	2
3	9.8	6	3
4	9.4	5	4

Step 3: Analyzing the Data

1. df.shape(): This function is used to understand the number of rows (observations) and columns (features) in the dataset. This gives an overview of the dataset's size and structure.

df.shape

Output:

(1143, 13)

2. df.info(): This function helps us to understand the dataset by showing the number of records in each column, type of data, whether any values are missing and how much memory the dataset uses.

df.info()

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1143 entries, 0 to 1142
```

```
Data columns (total 13 columns):
```

#	Column	Non-Null Count	Dtype
0	fixed acidity	1143 non-null	float64
1	volatile acidity	1143 non-null	float64
2	citric acid	1143 non-null	float64
3	residual sugar	1143 non-null	float64
4	chlorides	1143 non-null	float64
5	free sulfur dioxide	1143 non-null	float64
6	total sulfur dioxide	1143 non-null	float64
7	density	1143 non-null	float64
8	pH	1143 non-null	float64
9	sulphates	1143 non-null	float64
10	alcohol	1143 non-null	float64
11	quality	1143 non-null	int64
12	Id	1143 non-null	int64

```
dtypes: float64(11), int64(2)
```

```
memory usage: 116.2 KB
```

3. df.describe().T: This method gives a statistical summary of the DataFrame (Transpose) showing values like count, mean, standard deviation, minimum and quartiles for each numerical column. It helps in summarizing the central tendency and spread of the data.

df.describe().T

	count	mean	std	min	25%	50%	75%	max
fixed acidity	1143.0	8.311111	1.747595	4.60000	7.10000	7.90000	9.10000	15.90000
volatile acidity	1143.0	0.531339	0.179633	0.12000	0.39250	0.52000	0.64000	1.58000
citric acid	1143.0	0.268364	0.196686	0.00000	0.09000	0.25000	0.42000	1.00000
residual sugar	1143.0	2.532152	1.355917	0.90000	1.90000	2.20000	2.60000	15.50000
chlorides	1143.0	0.086933	0.047267	0.01200	0.07000	0.07900	0.09000	0.61100
free sulfur dioxide	1143.0	15.615486	10.250486	1.00000	7.00000	13.00000	21.00000	68.00000
total sulfur dioxide	1143.0	45.914698	32.782130	6.00000	21.00000	37.00000	61.00000	289.00000
density	1143.0	0.996730	0.001925	0.99007	0.99557	0.99668	0.997845	1.00369
pH	1143.0	3.311015	0.156664	2.74000	3.20500	3.31000	3.40000	4.01000
sulphates	1143.0	0.657708	0.170399	0.33000	0.55000	0.62000	0.73000	2.00000
alcohol	1143.0	10.442111	1.082196	8.40000	9.50000	10.20000	11.10000	14.90000
quality	1143.0	5.657043	0.805824	3.00000	5.00000	6.00000	6.00000	8.00000
Id	1143.0	804.969379	463.997116	0.00000	411.00000	794.00000	1209.50000	1597.00000

4. **df.columns.tolist():** This converts the column names of the DataFrame into a Python list making it easy to access and manipulate the column names.

```
df.columns.tolist()
```

Output:

```
['fixed acidity',  
'volatile acidity',  
'citric acid',  
'residual sugar',  
'chlorides',  
'free sulfur dioxide',  
'total sulfur dioxide',  
'density',  
'pH',  
'sulphates',  
'alcohol',  
'quality',  
'Id']
```

Step 4 : Checking Missing Values

df.isnull().sum(): This checks for missing values in each column and returns the total number of null values per column helping us to identify any gaps in our data.

```
df.isnull().sum()
```

Output:

	0
fixed acidity	0
volatile acidity	0
citric acid	0
residual sugar	0
chlorides	0
free sulfur dioxide	0
total sulfur dioxide	0
density	0
pH	0
sulphates	0
alcohol	0
quality	0
Id	0

dtype: int64

Step 5 : Checking for the duplicate values

df.duplicated().sum(): Returns the number of duplicate rows in the dataset.

```
df.duplicated().sum()
```

Output:

	0
fixed acidity	91
volatile acidity	135
citric acid	77
residual sugar	80
chlorides	131
free sulfur dioxide	53
total sulfur dioxide	138
density	388
pH	87
sulphates	89
alcohol	61
quality	6
Id	1143

dtype: int64

Step 6: Univariate Analysis

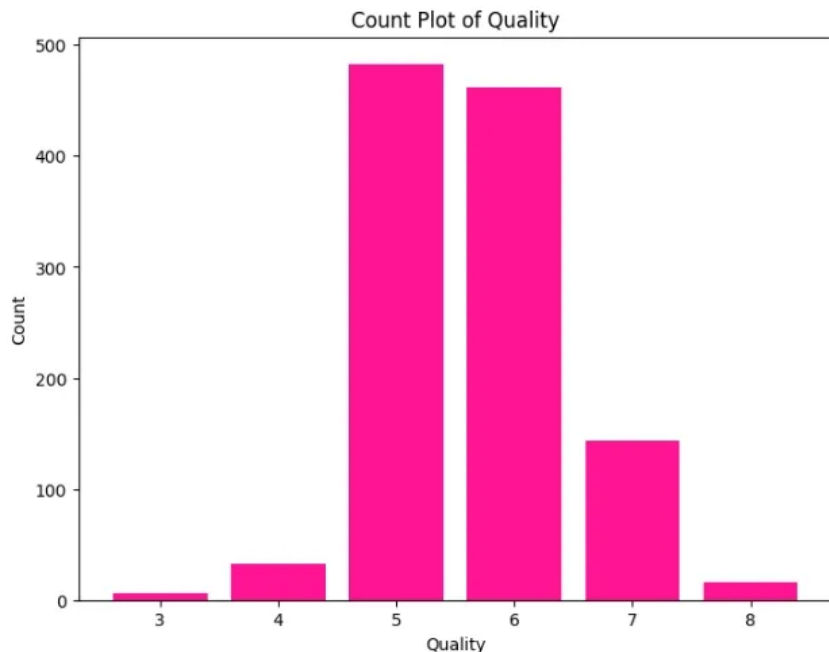
In [Univariate analysis](#) plotting the right charts can help us to better understand the data making the data visualization so important.

1. Bar Plot for evaluating the count of the wine with its quality rate.

```
quality_counts = df['quality'].value_counts()
```

```
plt.figure(figsize=(8, 6))  
plt.bar(quality_counts.index, quality_counts, color='deeppink')  
plt.title('Count Plot of Quality')  
plt.xlabel('Quality')  
plt.ylabel('Count')  
plt.show()
```

Output:



If you want to practice Bivariate and multivariate analysis

[Blog Link:](#)

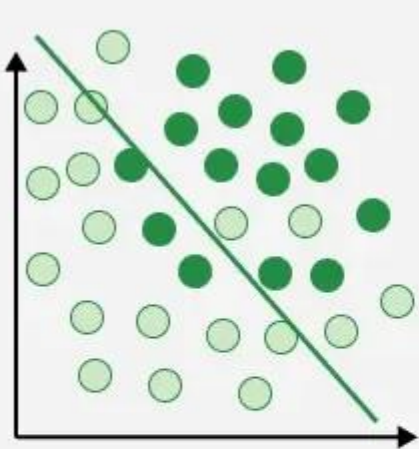
[EDA - Exploratory Data Analysis in Python – GeeksforGeeks](#)

Here, this count plot graph shows the count of the wine with its quality rate.

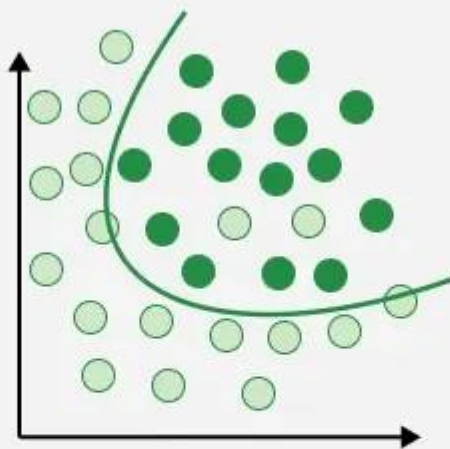
Regularization in Machine Learning

Regularization is a technique used in machine learning to prevent overfitting, which otherwise causes models to perform poorly on unseen data. By adding a penalty for complexity, regularization encourages simpler and more generalizable models.

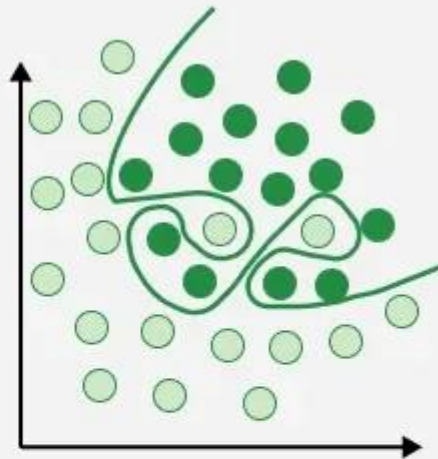
- **Prevents overfitting:** Adds constraints to the model to reduce the risk of memorizing noise in the training data.
- **Improves generalization:** Encourages simpler models that perform better on new, unseen data.



Under-fitting



Appropriate-fitting



Over-fitting

Types of Regularization

There are mainly 3 types of regularization techniques, each applying penalties in different ways to control model complexity and improve generalization.

1. Lasso Regression

A regression model which uses the L1 Regularization technique is called LASSO (Least Absolute Shrinkage and Selection Operator) regression. It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L). This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m |w_j|$$

Where

- m : Number of Features
- n : Number of Examples
- y_i : Actual Target Value
- $\hat{y}_i$: Predicted Target Value

2. Ridge Regression

A regression model that uses the L2 regularization technique is called Ridge regression. It adds the squared magnitude of the coefficient as a penalty term to the loss function(L). It handles multicollinearity by shrinking the coefficients of correlated features, reducing their variance and preventing any single feature from dominating the model.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m w_j^2$$

Where,

- n : Number of examples or data points
- m : Number of features i.e predictor variables
- y_i : Actual target value for the i th example
- $\hat{y}_i$: Predicted target value for the i th example
- w_i : Coefficients of the features
- λ : Regularization parameter that controls the strength of regularization

3. Elastic Net Regression

Elastic Net Regression is a combination of both L1 as well as L2 regularization. It combines both L1 (absolute values) and L2 (squared values) penalties on the coefficients. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1 - \alpha) \sum_{j=1}^m |w_j| + \alpha \sum_{j=1}^m w_j^2 \right)$$

Where

- n : Number of examples (data points)
- m : Number of features (predictor variables)
- y_i : Actual target value for the i^{th} example
- $\hat{y}_i$: Predicted target value for the i^{th} example
- w_i : Coefficients of the features
- λ : Regularization parameter that controls the strength of regularization
- α : Mixing parameter where $0 \leq \alpha \leq 1$ and $\alpha = 1$ corresponds to Lasso (L_1 regularization), $\alpha = 0$ corresponds to Ridge (L_2 regularization) and Values between 0 and 1 provide a balance of both L1 and L2 regularization

Benefits of Regularization

Now, let's see various benefits of regularization which are as follows:

- **Prevents Overfitting:** Regularization helps models focus on underlying patterns instead of memorizing noise in the training data.
- **Enhances Performance:** Prevents excessive weighting of outliers or irrelevant features helps in improving overall model accuracy.
- **Stabilizes Models:** Reduces sensitivity to minor data changes which ensures consistency across different data subsets.
- **Prevents Complexity:** Keeps model from becoming too complex which is important for limited or noisy data.
- **Handles Multicollinearity:** Reduces the magnitudes of correlated coefficients helps in improving model stability.
- **Promotes Consistency:** Ensures reliable performance across different datasets which reduces the risk of large performance shifts.